ArtiFrame

Projet de session - Équipe 16

Livrable 2 - Document de design

IFT-3100 – H24

Infographie

Travail présenté à M. Philippe Voyer

Charles Brassard – 536 783 602

Mathieu Chrétien – 536 875 935

Vincent Cousineau – 536 961 396

Anthony Veillet – 536 779 403

Table des matière:

Table des matière: 2
Sommaire 5
Interactivité 6
spacebar + i (ArtiFrame2D): Importer une image: 6
spacebar + u (ArtiFrame2D): Layer up rapide: 6
spacebar + d (ArtiFrame2D): Layer down rapide: 6
DEL + a (ArtiFrame2D): Suppression total: 6
keyReleased ‘s’ (ArtiFrame2D): Dessiner une primitive carré: 6
keyReleased ‘r’ (ArtiFrame2D): Dessiner une primitive rectangle: 6
keyReleased ‘c’ (ArtiFrame2D): Dessiner une primitive cercle: 6
Cette touche à l’intérieur du mode 2D permet de faire le dessin automatique d’un cercle 6
keyReleased ‘e’ (ArtiFrame2D): Dessiner une primitive ellipse: 6
keyReleased ‘p’ (ArtiFrame2D): Dessiner un polygone régulier: 6
keyReleased ‘w’ (ArtiFrame2D): Dessiner une flèche: 6
keyReleased ‘q’ (ArtiFrame2D): Dessiner une étoile: 6
keyReleased ‘DEL’ (ArtiFrame2D): Entrer/sortir en mode suppression: 7
keyRealased ‘u’ (ArtiFrame2D): Layer up: 7
keyRealased ‘d’ d (ArtiFrame2D): Layer down: 7
keyReleased ‘m’ (ArtiFrame2D): Entrer/sortir en mode exportation: 7
keyReleased ‘t’ (ArtiFrame2D): Entrer/sortir en mode rotation: 7
W, A, S, D, Q, E (ArtiFrame3D): Bouger la caméra 7
Clic droit, scroll (ArtiFrame3D): Zoom 7
Clic gauche (ArtiFrame3D): Rotation de la caméra autour du modèle 7
Double-clic (ArtiFrame3D): Réinitialiser la caméra à l’état initiale 7
1,2,3 (ArtiFrame3D): Bouger les positions des lumières 7
Technologie 8
Compilation 9
Architecture 10
Arborescence des fichiers du projet: 10
Section 2D: Section 3D: Section UI et main : 10
Diagramme de classe: 11
Fonctionnalités 12
1. Image 12
1.1 Importation d’images 12
1.2 Exportation d’images 12
1.3 Échantillonnage d’images 12
1.4 Espaces de couleur 12
1.5 Histogramme 12
2. Dessin vectoriel 13
2.1 Curseur dynamique 13
2.2 Outils de dessin 13

2.3 Primitives vectorielles.....14
2.4 Formes vectorielles.....14
2.5 Interface.....14
3. Transformation.....15
3.1 Graphe de scène.....15
3.2 Sélection multiple.....15
3.3 Transformations interactives.....15
4. Géométrie.....15
4.1 Boîte de délimitation.....15
4.2 Primitives géométriques.....16
4.3 Modèles 3D.....16
4.4 Animation.....16
4.5 Instanciation.....16
5. Caméra.....17
5.1 Point de vue.....17
5.2 Mode de projection.....17
6. Texture.....17
6.1 Coordonnées de texture.....17
6.2 Filtrage.....17
6.3 Mappage tonal.....17
6.5 Texture procédurale.....18
7. Illumination classique.....18
7.1 Modèles d'illumination.....18
7.2 Matériaux.....18
7.3 Types de lumière.....18
7.4 Lumières multiples.....18
8. Topologie.....19
8.1 Courbe paramétrique.....19
8.2 Surface paramétrique.....19
8.3 Shader de tessellation.....19
8.4 Triangulation.....20
8.5 Effet de relief.....20
9. Lancer de rayon.....20
9.1 Intersection.....20
9.2 Réflexion.....20
9.3 Réfraction.....21
9.4 Ombrage.....21
9.5 Illumination globale.....21
10. Illumination moderne.....21
10.3 Métallicité.....21
10.4 Microfacettes.....21
Ressources.....22
Cursors:.....22
Shader:.....22

Présentation..... 23
Division des tâches:..... 24
Rapport 1:.....24
Rapport 2:.....25

Sommaire

Le premier livrable réalisé vise principalement à élaborer une application dédiée à la manipulation en temps réel d'images et de modèles tridimensionnels. Cette application a été élaborée en se servant du C++ comme langage de programmation, appuyée par des environnements de développement tels que Visual Studio 2022. Il s'appuie sur des librairies comme OpenGL et OpenFrameworks. Plusieurs bibliothèques pouvaient être utilisées par les équipes. Nous avons pour notre part utilisé la bibliothèque ofxDatGui a été exploitée par l'équipe.

Ce projet se divise en plusieurs segments, traitant chacun d'un aspect particulier de l'application. Le premier segment met l'accent sur les capacités interactives couplées à des fonctionnalités de traitement d'images, incluant l'exportation, l'importation, la suppression, la rotation, le déplacement, et le zoom des images, en plus de la rotation des modèles et de la réinitialisation de la caméra.

Le second segment porte sur les technologies employées pour la mise au point de l'application. Pour sa création, l'équipe a utilisé Visual Studio 2022, ainsi que les librairies OpenGL, openFrameworks, et ofxDatGui pour l'élaboration de l'interface utilisateur. Ce segment détaille également les prérequis pour compiler le projet, y compris l'installation d'openFrameworks et de ofxDatGui.

Le troisième segment aborde l'architecture de l'application, décrivant la structure des fichiers et dossiers et les diagrammes de classes conçus pour le développement de l'application.

Le quatrième segment est consacré aux fonctionnalités implémentées dans l'application. L'équipe a intégré diverses fonctionnalités comme l'importation et l'exportation d'images, l'échantillonnage d'images, la gestion de l'espace couleur, les histogrammes, un curseur dynamique, des outils de dessin, ainsi que des primitives et formes vectorielles. Une description détaillée de l'interface utilisateur est également fournie, incluant le graphe de scène, la sélection multiple, les transformations interactives, l'historique des transformations, la boîte de délimitation, les primitives géométriques, les modèles 3D, l'animation, les shaders, ainsi que les perspectives et modes de projection de la caméra.

La dernière partie du projet met en lumière les ressources mobilisées pour le développement de l'application et détaille les contributions spécifiques de chaque membre de l'équipe.

Interactivité

spacebar + i (ArtiFrame2D): Importer une image:

La combinaison de ces deux touches permettent l'importation d'une image à partir de l'explorateur de fichier.

spacebar + u (ArtiFrame2D): Layer up rapide:

Permet de rapidement faire monter un objet dans le graph de scène.

spacebar + d (ArtiFrame2D): Layer down rapide:

Permet de rapidement faire descendre un objet dans le graph de scène.

DEL + a (ArtiFrame2D): Suppression total:

Permet de rapidement supprimer tous les objets dans le graph de scène.

keyReleased 's' (ArtiFrame2D): Dessiner une primitive carré:

Cette touche à l'intérieur du mode 2D permet de faire le dessin automatique d'un carré.

keyReleased 'r' (ArtiFrame2D): Dessiner une primitive rectangle:

Cette touche à l'intérieur du mode 2D permet de faire le dessin automatique d'un rectangle.

keyReleased 'c' (ArtiFrame2D): Dessiner une primitive cercle:

Cette touche à l'intérieur du mode 2D permet de faire le dessin automatique d'un cercle.

keyReleased 'e' (ArtiFrame2D): Dessiner une primitive ellipse:

Cette touche à l'intérieur du mode 2D permet de faire le dessin automatique d'une ellipse.

keyReleased 'p' (ArtiFrame2D): Dessiner un polygone régulier:

Cette touche à l'intérieur du mode 2D permet de faire le dessin automatique d'un polygone régulier.

keyReleased 'w' (ArtiFrame2D): Dessiner une flèche:

Cette touche à l'intérieur du mode 2D permet de faire le dessin automatique d'une flèche.

keyReleased 'q' (ArtiFrame2D): Dessiner une étoile:

Cette touche à l'intérieur du mode 2D permet de faire le dessin automatique d'une étoile.

keyReleased ‘DEL’ (ArtiFrame2D): Entrer/sortir en mode suppression:

Permet d’entrer ou de sortir du mode suppression. Une fois dans le mode, faire un clique gauche de la souris pour supprimer l’objet sélectionné.

keyRealased ‘u’ (ArtiFrame2D): Layer up:

Permet de faire monter un objet dans le graph de scène, un par un.

keyRealased ‘d’ d (ArtiFrame2D): Layer down:

Permet de faire descendre un objet dans le graph de scène, un par un.

keyReleased ‘m’ (ArtiFrame2D): Entrer/sortir en mode exportation:

Permet d’entrer/sortir en mode exportation pour exporter l’image d’une scène. Une fois dans le mode, faire un clique gauche de la souris pour exporter.

keyReleased ‘t’ (ArtiFrame2D): Entrer/sortir en mode rotation:

Permet d’entrer/sortir en mode de rotation, lorsqu’un élément est sélectionné. Une fois dans le mode, faire un clique gauche de la souris pour effectuer une rotation de 90 degrés.

W, A, S, D, Q, E (ArtiFrame3D): Bouger la caméra

La fonctionnalité de bouger la caméra dans la scène est implanté avec l'aide de plusieurs touches du clavier. Les touches W et A bougent la caméra sur l’axe x alors que le Q et E contrôle l’axe y. Finalement, la profondeur(axe de z) est contrôlée par les touches S et W.

Clic droit, scroll (ArtiFrame3D): Zoom

Il est possible de contrôler la caméra sur l'axe des z avec soit le click droit ou la molette de la souris.

Clic gauche (ArtiFrame3D): Rotation de la caméra autour du modèle

Le clic gauche permet de faire une rotation de la caméra autour du point central de la scène.

Double-clic (ArtiFrame3D): Réinitialiser la caméra à l’état initiale

Un double clic gauche de la souris permet de réinitialiser la caméra à sa position initiale.

1,2,3 (ArtiFrame3D): Bouger les positions des lumières.

Il est possible de bouger les positions des lumières grâce aux touches 1,2 et 3.

Technologie

Au cours de la réalisation de ce livrable, nous avons utilisé différentes technologies pour concevoir une application graphique de qualité et efficace dans la réalisation des fonctionnalités demandées. Notre projet est implémenté en c++, ce qui était l'idéal étant donné la facilité et la connaissance des membres de l'équipe avec ce langage de programmation.

En ce qui concerne l'environnement de développement utilisé, nous avons opté pour VIsual Studio 2022. En plus d'être recommandé par le professeur, cet environnement était bien connu de l'un des membres de l'équipe et simplifiait la programmation, le débogage et l'intégration de bibliothèques nécessaires pour le développement d'un tel projet.

Nous avons notamment utilisé la bibliothèque ofxDatGui pour l'implémentation de notre interface utilisateur pour sa qualité visuel et utilisation relativement facile. De plus, étant donné les informations apprises durant le cours ainsi que pour son efficacité, nous avons utilisé les bibliothèques openGL et openFrameworks dans les différentes classes composant l'arborescence de notre projet. Ces bibliothèques nous ont permis de rendre beaucoup plus facile et rapide le développement de notre rendu graphique.

Dans notre section de l'application qui s'occupe du rendu graphique en 3D, nous utilisons GLSL. Notre utilisation de ce OpenGl Shading Language à grandement améliorer l'apparence de nos rendus, elle sera plus détaillée dans la section des fonctionnalités.

En bref, notre application à été réalisé en langage de programmation c++ et utilise des librairies lié à l'infographie comme OpenGl, openFrameworks et ofxDatGui pour améliorer la qualité du code, la qualité de l'application et l'efficacité des membres de l'équipe. Ces éléments combinés à l'environnement de développement Visual Studio 2022 nous a permis de concevoir une application aux multiples fonctionnalités et intuitives pour les utilisateurs potentiels.

Compilation

1. En premier lieu, télécharger l'archive fournis et la décompresser. Un version exécutable a été ajoutée dans la remise avec un raccourci pour éviter de devoir compiler le projet soit même. Si vous désirez compiler le projet a partir du code il faudra prendre les dossier /bin et /src et continuer à l'étape 2 avec ces dossiers dans votre dossier de travail(..\OpenFramework\apps\myApps\projet).

2. Par la suite, il faut s'assurer d'avoir openFrameworks et ofxDatGui d'installer sur votre système. Si jamais ce n'est pas le cas, il est possible d'aller installer openFrameworks à cette adresse https://openframeworks.cc/download/. Celle-ci permet l'installation de ofxDatGui https://github.com/braitsch/ofxDatGui. Une fois leurs installation confirmé ou effectué, il faut ouvrir le dossier 'addons' de openFrameworks et déposer le dossier 'ofxbraitsch' dans le dossier data du projet. Le dossier déposer permettra l'utilisation de ofxDatGui dans le projet.

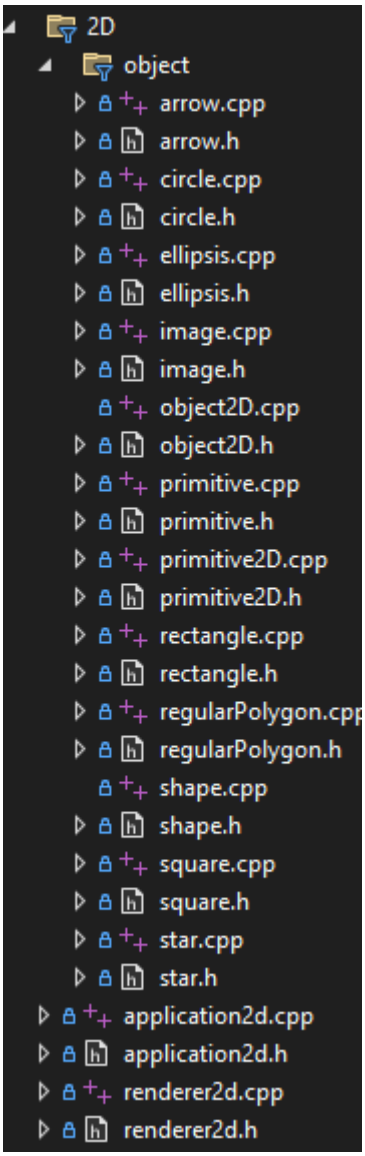
3. Nous pouvons maintenant générer le projet. Pour ce faire, entrer dans le dossier 'projetGenerator' de openFrameworks et lancé l'exécutable 'projectGenerator.exe'. Allez sélectionner l'archive du projet fourni à l'endroit déposé et ajoutez les addons 'ofxAssimpModelLoader', 'ofxGui' et 'ofxDatGui'. Préférablement choisir d'utiliser Visual Studio 2022 pour le projet étant donné que c'est l'environnement utilisé pour le développement.Il faut maintenant appuyer sur update pour générer le projet selon les configurations de votre système d'exploitation.

4. Finalement, il est nécessaire de supprimer les deux fichier ofApp.cpp et ofApp.h qui sont générés automatiquement dans openFrameworks.
5. Il faut aller dans les propriétés du projet => C/C++ => Langage et activer la prise en charge OpenMP (mettre à "Oui(/openmp)").

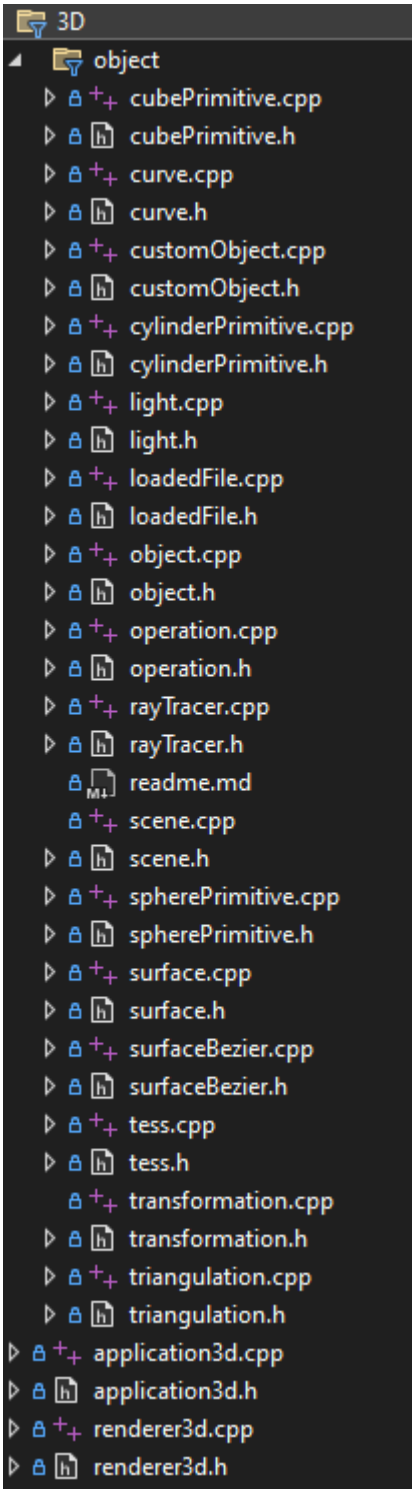
Architecture

Arborescence des fichiers du projet:

Section 2D:



Section 3D:



Section UI et main :

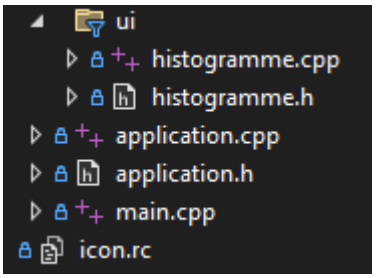
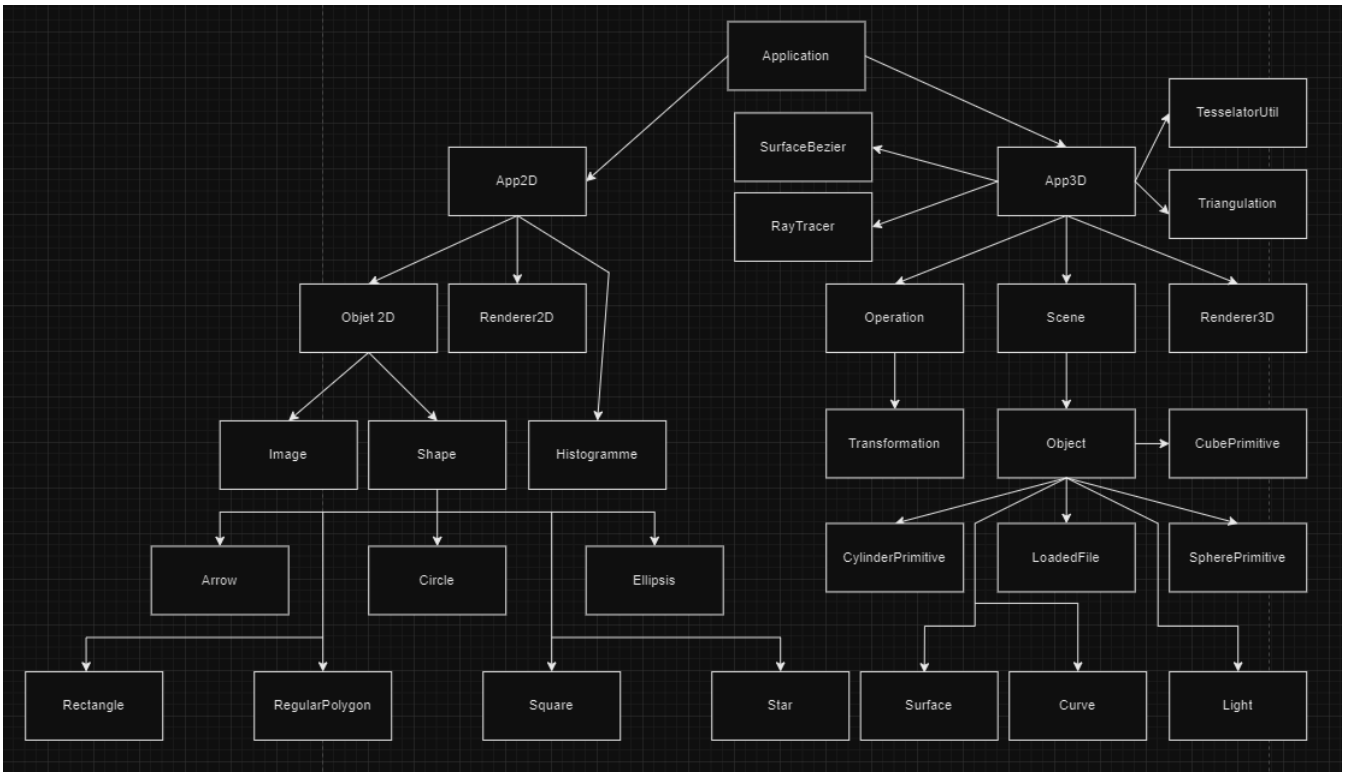


Diagramme de classe:



Fonctionnalités

1. Image

1.1 Importation d'images

L'application supporte l'importation d'images via le bouton "File" et ensuite "Import". Les types de fichiers que l'on peut importer sont PNG, JPEG et GIF.

1.2 Exportation d'images

L'application supporte l'exportation d'images via le bouton "File" et ensuite "Export". Lorsque l'on export, une fenêtre d'exportation s'ouvre nous permettant de décider où l'on veut mettre les images exportées et on confirme le tout.

1.3 Échantillonnage d'images

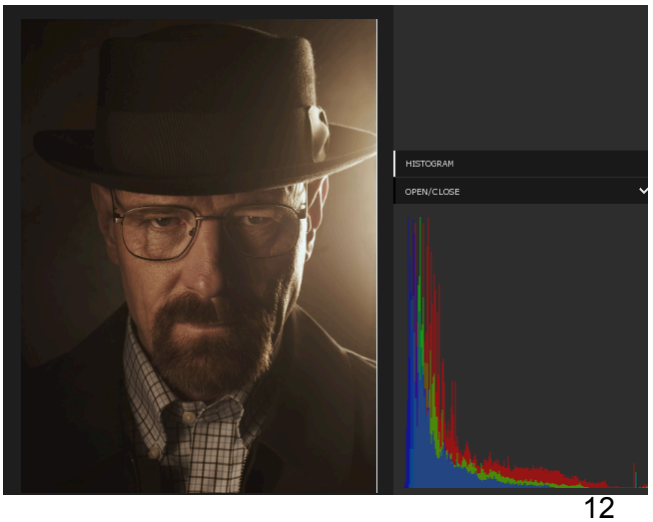
L'application supporte l'échantillonnage d'images et permet d'importer des images pour en faire une nouvelle en faisant un agencement avec d'autres images. On peut également sélectionner chaque image et décider sur quelle couche ils sont en utilisant les boutons "Layer Up" et "Layer Down" dans le menu "Edit".

1.4 Espaces de couleur

L'interface de l'application permet de modifier l'espace de couleur dans la représentation 2D. Les valeurs HSB (Hue, Saturation, Brightness) et RGB (Red, Green, Blue) sont automatiquement ajustées en fonction des modifications apportées à l'une ou l'autre. La sélection de la couleur se fait avant le rendu des éléments à l'écran. Ainsi, si on génère une forme initialement blanche, sa couleur se transformera en celle choisie via le menu de l'interface situé à gauche. En revanche, pour appliquer la couleur de remplissage présente dans le menu des formes, il suffit d'ajuster la teinte vers le blanc.

1.5 Histogramme

L'affichage de l'histogramme est disponible dans le coin inférieur droit de l'application. L'histogramme affiché est celui de l'image qui est sélectionnée. Celui-ci est dessiné lors de l'ouverture du dossier histogramme et permet de visualiser graphiquement les proportions des couleurs rouge, bleu et vert des pixels au sein de l'image.



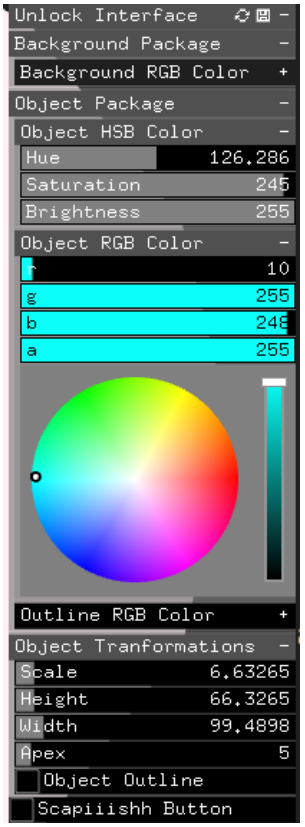
2. Dessin vectoriel

2.1 Curseur dynamique

Le curseur évolue en fonction des interactions de l'utilisateur. L'application propose cinq types de curseurs différents. Initialement, le curseur est normal, affiché lorsque aucune action spécifique n'est entreprise. Ensuite, il se transforme lors du déplacement (drag) d'une image, de la suppression (“delete”) d'une image, de la rotation (“t”) d'une image(lorsqu’une image est sélectionnée), et enfin lors de l'exportation(“m”). Ces changements de curseur sont uniquement visibles dans la vue 2D. La rotation se fait à 90 degrés, donc il est normal qu' on le voit pas sur le carré, puisqu’il reste pareil après la rotation.

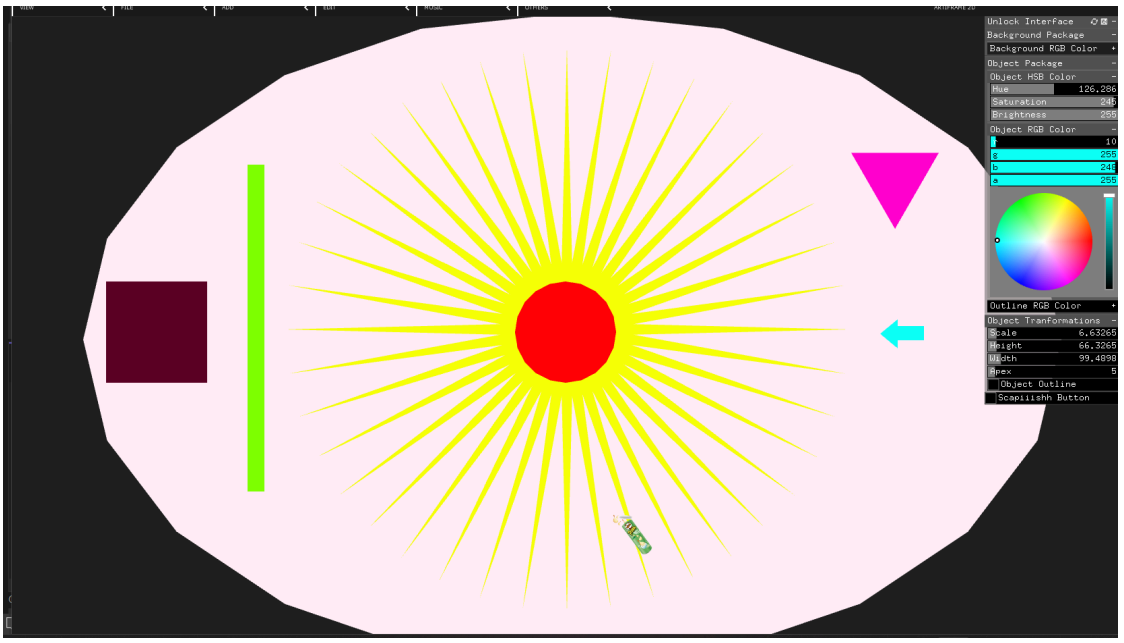
2.2 Outils de dessin

Quand on crée ou choisit une forme, un menu dédié s'affiche pour ajuster les caractéristiques de cette forme. Cela inclut la modification de ses dimensions et couleurs. Par contre, le paramètre du nombre de sommets n'affecte que l’étoile et le polygone. Il est également faisable de changer la couleur de fond via le menu du haut “background color”.



2.3 Primitives vectorielles

L'application offre la possibilité de générer cinq formes vectorielles de base : un carré, un rectangle, un cercle, une ellipse et un polygone régulier. Chacune de ces formes peut être modifiée à l'aide du menu de transformation qui s'affiche lors de leur création.



2.4 Formes vectorielles

L’application permet de créer deux formes vectorielles : un étoile et une flèche. Ils peuvent être modifiés de la même façon que les primitives vectorielles à l’aide du menu de transformation.

2.5 Interface

L’interface comporte une barre “header” située sur le haut de l’application permettant d’accéder aux fonctionnalités de l’application. Il y a également une fenêtre “pop-up” permettant de voir les outils de transformations. Sur la droite, il y a une colonne permettant de voir les objets présents dans la scène et les sélectionnés. Puis en dessous de cette colonne se situe l’histogramme qui s’ouvre ou se ferme.

3. Transformation

3.1 Graphe de scène

Au sein de l'application, un graphe de scène est employé pour organiser et dessiner les éléments dans l'espace 3D, tandis que les éléments 2D sont gérés dans un vecteur qui définit l'ordre des différentes couches du dessin. Ce graphe de scène inclut des objets, qui peuvent être soit des fichiers importés, soit des formes de base générées par l'application.
Chaque objet est doté d'un parent et peut avoir plusieurs enfants, permettant ainsi à la scène de regrouper tous les objets à partir de sa racine. Lors du dessin, l'appel est d'abord fait aux objets racines, qui appellent ensuite leurs enfants, lesquels sont dessinés en respectant les transformations appliquées par leur parent. Pour ajouter un nouvel objet en tant qu'enfant à un objet existant, il suffit de le créer ou de l'ajouter alors qu'un objet est déjà sélectionné. La sélection, qui est visible dans la liste située en dessous du header, indique quels items subiront les transformations. Les objets peuvent être identifiés dans la vue 3D par les boîtes qui les entourent.

3.2 Sélection multiple

L’application supporte la sélection de plusieurs objets présents dans une scène et on peut modifier les valeurs de certains attributs communs à tous les éléments sélectionnés à l'aide du menu de transformation. La sélection multiple est seulement disponible dans la vue 3D. Les objets sélectionnés peuvent être identifiés à partir de la liste de sélection actuelle qui apparaît vers le milieu du haut de l’écran(en dessous de la barre de folders d’outils), ainsi que par les boîtes encombrantes qui les entourent. Pour sélectionner ou désélectionner un élément, il suffit de cliquer sur son nom dans la liste située à droite de l'écran.

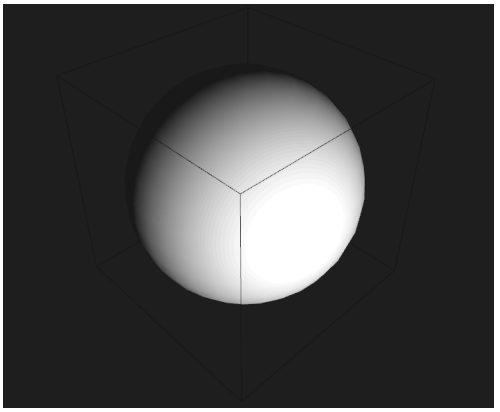
3.3 Transformations interactives

Trois types de transformations (translation, rotation et proportion) peuvent être effectués en utilisant le menu de transformation. On doit sélectionner le type de transformation souhaité, l'axe désiré, puis appuyer sur le bouton "Apply". On peut également choisir le nombre de côtés pour les polygones et les étoiles.

4. Géométrie

4.1 Boîte de délimitation

Chaque objet 3d qui est ajouté dans la scène et dont l’utilisateur a sélectionné est affiché dans une boîte autour de l’objet. Cette boite est juste assez grande pour s’assurer que tous les sommets du modèle 3d sont à l’intérieur.



4.2 Primitives géométriques

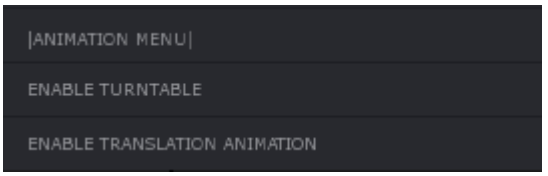
Il est possible d'ajouter un total de trois primitives géométriques, soit un cube, une sphère et un cylindre. Dans l'application le fichier "ADD", qui est disponible dans le header en haut de l'écran, peut être déployé quand appuyer pour montrer les différents choix.

4.3 Modèles 3D

Il est possible d'ajouter le modèle 3d de Suzanne la mascotte de blender dans le menu de création de des primitives géométriques. De plus, il est possible d'importer des modèle 3d externe à l'aide de la librairie ofAssimpModelLoader qui peut importer des fichiers de plusieurs type dont Blender 3D (.blend), 3D Studio Max (.3ds), Wavefront Object (.obj), Autodesk FBX (.fbx) et STereoLithography (.stl).

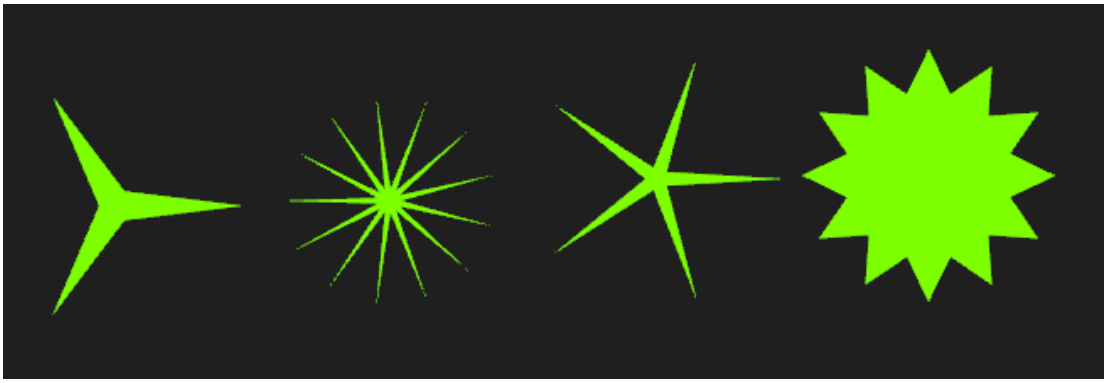
4.4 Animation

Les objets sélectionnés peuvent être animés de deux façons différentes. Le premier mode du tourne-disque (Turntable) permet de faire une rotation du modèle par rapport au centre de la scène. Le deuxième mode d'animation de translation fait bouger le modèle sur l'axe des X dans une boucle d'aller-retour.



4.5 Instanciation

Plusieurs instances d'étoiles et de polygones réguliers différents peuvent être créées dans le programme grâce au nombre d'apex, qui représente le nombre de points utilisés pour créer la forme. En changeant également la hauteur et la largeur, les étoiles peuvent être modifiées pour avoir une apparence spéciale voulue.



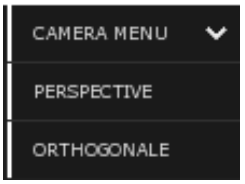
5. Caméra

5.1 Point de vue

La vue 3D est équipée d'une caméra permettant de se déplacer dans toutes les directions afin d'observer les objets 3D sous l'angle désiré. Vous pouvez vous déplacer en utilisant les touches w, a, s et d, ainsi que les touches q et e pour monter et descendre. De plus, la caméra vous permet de vous rapprocher des objets en utilisant la molette de la souris et de faire pivoter autour du modèle. En cliquant au centre, la caméra pivote autour de l'axe vertical, tandis qu'un clic plus éloigné du centre permet de pivoter autour de l'axe reliant la caméra à son point focal. En double-cliquant, vous pouvez recentrer la caméra en cas de perte de repères.

5.2 Mode de projection

L'application supporte deux modes de projection (perspective et orthogonale). Pour changer il suffit d'aller dans le "Camera Menu" et choisir sa vue. En ouvrant l'application, la vue est en mode projection en perspective.



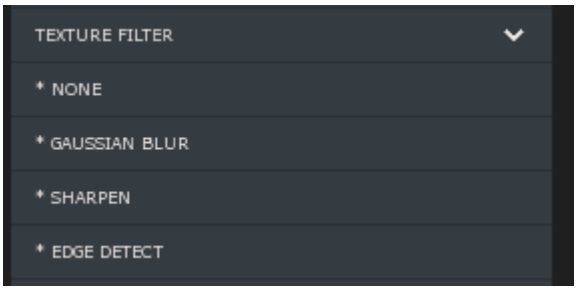
6. Texture

6.1 Coordonnées de texture

Les textures 2D sont appliquées aux différents objets de la scène dans les shader ou les texture sont supporté dont toneMapping (affichage de texture diffuse seulement sans lumiere pris en compte) et PBR.

6.2 Filtrage

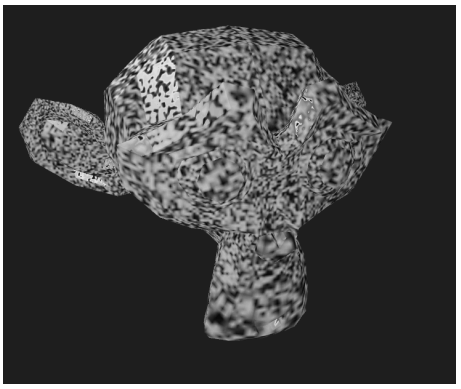
Il est possible de choisir 3 différents types de filtres de texture d'objet. Il suffit de sélectionner le mode de rendu Texture ou PBR, et d'ajouter une texture diffuse à l'objet pour ensuite lui appliquer un filtre. Voici les 3 filtres différents proposés par le logiciel: le gaussian blur, le edge detect et le sharpen.



6.3 Mappage tonal

Le programme permet de modifier le ton des textures diffuse à l'aide des sliders de la section Tone Mapping du menu des textures. Tout comme les filtres de texture, il faut sélectionner un mode de rendu ou la texture peut être observée.

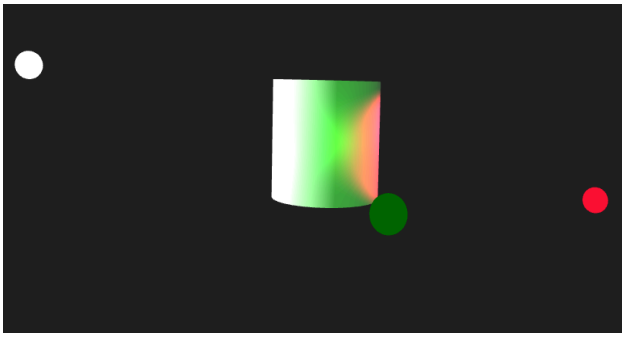
6.5 Texture procédurale



Le menu “File” comporte un bouton spécial qui vous permet de créer une texture diffuse générée aléatoirement à l’aide de bruit Perlin.

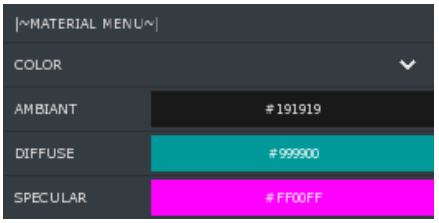
7. Illumination classique

7.1 Modèles d’illumination



Notre application permet d’obtenir un rendu selon trois différents modèles d’illumination, soit le modèle de Lambert, le modèle de Phong et le modèle de Blinn-Phong. Chacun de ces trois modèles offerts supporte quatre différents types de lumières, ceux-ci seront présentés à la section *7.3*. Pour sélectionner un modèle d’illumination, il faut d’abord entrer dans la vue d’édition 3D, puis ensuite ouvrir le menu déroulant “render mode” et sélectionner le modèle désiré. Finalement, il est également possible de déplacer les lumières grâce aux touches “1” “2” et “3” du clavier. Appuyer sur ces touches déplacera la lumière qui leur est liée à l’emplacement du curseur de la souris. Il est important de noter que l’emplacement de l’objet dans l’espace n’est pas pris en compte par l’éclairage, il est donc recommandé de maintenir l’objet au centre de l’écran pour bien observer l’effet des shaders.

7.2 Matériaux



On peut choisir les spécificités de notre matériau grâce à la section “Material Menu” de notre menu. Les parties liées à cette feature sont la possibilité de modifier la couleur des réflexions ambiantes, spéculaires et diffuses présentes sur l’objet de la scène.

7.3 Types de lumière

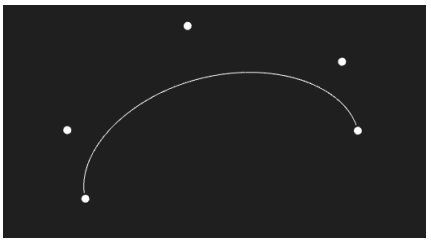
On retrouve dans notre application les quatres différents types de lumières, soit une lumière ambiante, une lumière de type projecteur, une lumière directionnelle et une lumière en point.

7.4 Lumières multiples

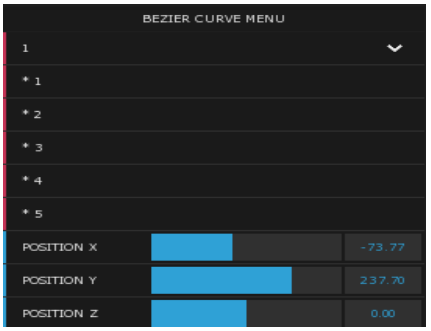
Comme mentionné plus tôt, chacune de nos lumières peuvent être déplacées grâce aux touches “1”, “2” et “3” du clavier. On a également des panneaux pour décider de la couleur de ces lumières et des sliders pour ajuster leur intensité selon ce que l’on désire.

8. Topologie

8.1 Courbe paramétrique



Notre application permet de créer une courbe paramétrique de bézier avec cinq points de contrôle. La création d'une courbe peut se faire dans le menu déroulant "Add" et ensuite sélectionner "Add bezier curve". Une fois la courbe créée, un menu lié à cette courbe apparaît et on peut sélectionner les différents points de contrôle et les déplacer en x, y et z.

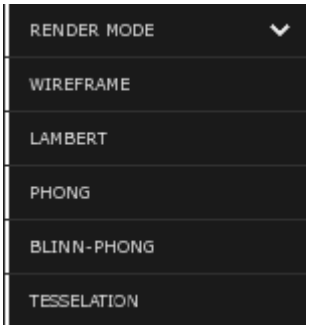


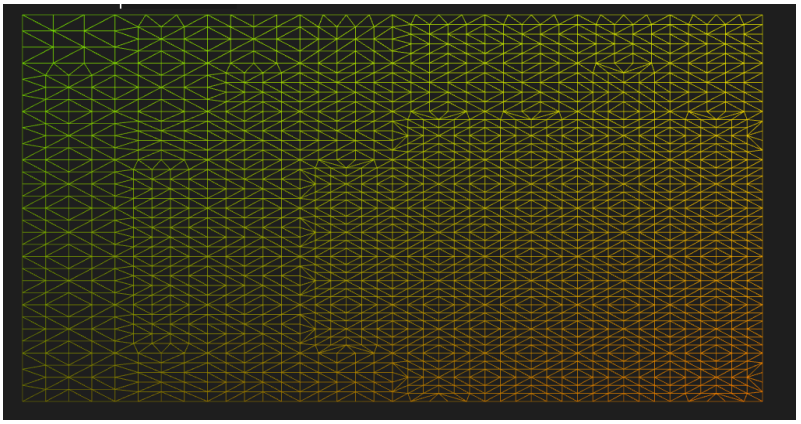
8.2 Surface paramétrique

Similairement à l'ajout de courbe, on peut ajouter une surface paramétrique avec un nombre de points de contrôle fixe (8) dont leurs positions en axe Y donne un effet de courbure qui est désiré. Le bouton pour l'ajout se trouve juste en dessous de celui pour l'ajout d'une courbe et un menu spécifique à la surface apparaît à sa création, ce menu offre des fonctionnalités similaires à celui de la courbe.

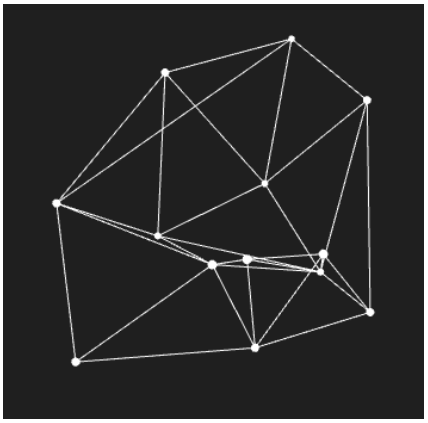
8.3 Shader de tessellation

On peut utiliser l'option "Tessellation" du menu déroulant "Render mode" pour voir un shader de tessellation appliqué sur une surface. Les objets présents dans le rendu disparaissent et laissent place à un mesh qui change dynamiquement selon la position de la souris sur l'écran.





8.4 Triangulation

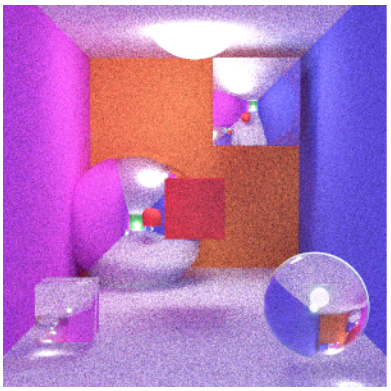


Notre application permet de placer des vertex selon l'algorithme de Delaunay pour faire la création d'objets customisés. Pour accéder à ce mode, il faut d'abord entrer dans le menu déroulant "Add" puis sélectionner l'option "Add custom object". Par la suite, il faut faire des clics droit sur la souris pour ajouter des vertex à l' endroit désiré. Il est important de noter qu'il faut un minimum initial de trois clics droits pour que l'objet apparaisse, étant donné que la triangulation nécessite logiquement un minimum de trois points pour que le premier triangle soit formé. Les points sont à une distance fixe de la caméra, le fait de zoomer et dézoomer peut donc permettre un positionnement des points plus précis.

8.5 Effet de relief

L'application permet d'utiliser des normal maps en mode de rendu PBR. Pour ajouter une normal map aux objets sélectionnés, il faut accéder au menu déroulant "File" et cliquer sur "Import Normal Map" dans le menu des textures. Il est important de noter que vous devez importer une texture avec le l'option "Import Albedo texture" avant cela, faute de quoi l'objet s'affichera en noir.

9. Lancer de rayon



L'application supporte le raytracing. Pour lancer un rendu, il faut aller dans le menu "Render Mode" et cliquer sur le bouton "Ray Tracing". Cela lance un rendu en arrière-plan. Le rendu comporte seulement les objets importés et les affiche dans une boîte de Cornell. Le résultat est un fichier .ppm qui se trouve à la racine du projet.

9.1 Intersection

Le raytracing peut s'utiliser avec n'importe quel modèle 3D, car l'application supporte toutes les géométries. Par contre, le temps d'exécution est long, surtout pour les ordinateurs avec de moins bonnes cartes graphiques. Il est donc suggéré d'importer des formes ayant peu de polygones pour tester le raytracing. L'application supporte aussi le rendu des sphères et des cubes.

9.2 Réflexion

L'application supporte le rendu pour les objets possédant de la réflexion comme on voit la sphère miroir dans la scène.

9.3 Réfraction

L’application supporte le rendu pour les objets possédant de la réfraction comme on voit le cube en verre dans la scène.

9.4 Ombrage

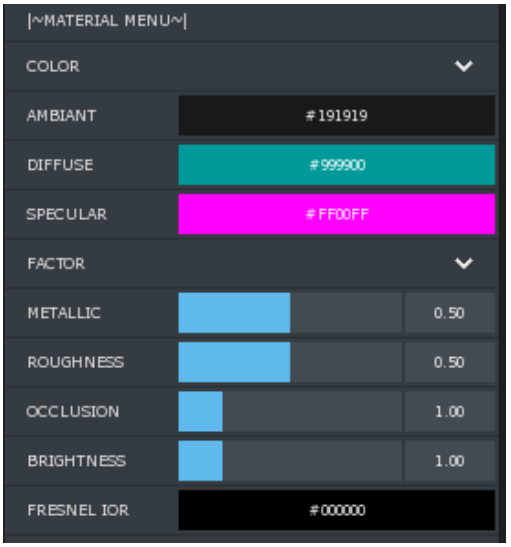
Le rendu du raytracing supporte les ombres comme on peut voir dans la scène.

9.5 Illumination globale

Le rendu du raytracing supporte l’illumination globale.

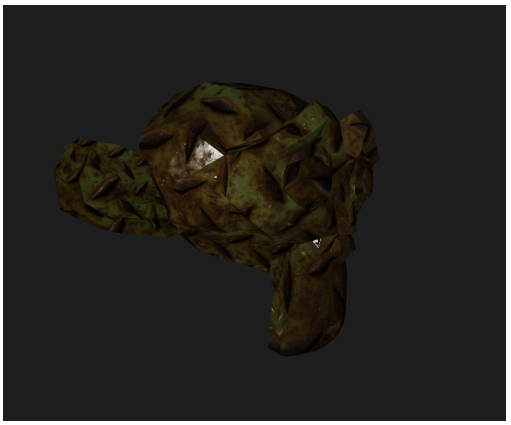
10. Illumination moderne

Le mode de rendu PBR de l'application exploite une variante ajustée du shader vu en classe pour réaliser le rendu des éléments en utilisant des textures simulant diverses propriétés des matériaux. Les caractéristiques des matériaux peuvent être ajustées via le menu des matériaux. De plus, la texture diffuse peut être modifiée à travers la section de Tone Mapping dans le menu des textures.



10.3 Métallicité

L’application supporte les metallic maps dans le mode de rendu PBR. Il suffit d’importer la metallic map avec le bouton “Import metallic map” dans le menu “File” pour ajouter la metallic map à la sélection actuelle.



10.4 Microfacettes

L’application supporte les roughness map dans le mode de rendu PBR. Il suffit d’importer la roughness map avec le bouton “Import roughness map” dans le menu “File” pour ajouter la roughness map à la sélection actuelle.

Ressources

Cursors:

Plusieurs différentes images ont été utilisées dans l'application pour faire état de curseur selon les différentes état du programme. Ces images proviennent de plusieurs sources externes qui offrent des curseurs gratuits à télécharger.
https://ui.blender.org/icons
https://www.flaticon.com/free-icons/import
https://www.flaticon.com/free-icons/export
https://custom-cursor.com/en

Shader:

Les shaders contrôlent la manière dont les objets 3D sont rendus à l'écran en les exécutant sur la carte graphique. Nous avons utilisé celui qui a été présenté en cours, c'est-à-dire le shader lambert_330_fs.glsl. Un shader de type "Lambert" est spécifiquement conçu pour simuler l'interaction de la lumière avec les surfaces opaques. En fonction de l'orientation par rapport à la source de lumière, il utilise une méthode d'ombrage diffuse pour évaluer la luminosité des surfaces.
https://github.com/philvoyer/IFT3100H24/tree/master/module04/EX06/IFT3100H24_LambertTeapot/bin/data

Les opérations mathématiques en lien avec le code des surfaces bézier viennent de cette extension : https://github.com/charlesveasey/ofxBezierSurface
L'extension n'a pas été utilisée mais on a utilisé du code de celle-ci dans notre implémentation.

L'algorithme pour la triangulation provient en partie de cette extension, de laquelle on a utilisé certaines parties du code pour nous assister. Voici l'extension :
https://github.com/obviousjim/ofxDelaunay

La texture métallique utilisée pour démontrer le normal mapping provient du cours.

Les textures de terrains proviennent de poly haven.

Présentation

Nous sommes 4 membres dans l'équipe:
● Anthony Veillet
● Charles Brassard
● Mathieu Chrétien
● VIncent Cousineau

- Bonjour, je m'appelle Anthony et je suis actuellement inscrit au programme de BAC en génie informatique. Le cours d'infographie figurait dans mes choix de cours à option. L'un de mes collègues de travail l'a suivi en hiver 2022 et m'en a parlé. Ses explications ont attiré ma curiosité et je me suis donc inscrit. Je possède quelques bases en infographie et en modélisation 3D, et ce cours semble super pertinent.

- Bonjour, je m'appelle Charles et je suis au bac intégré en informatique et gestion. Étant originaire du Bas-Saint-Laurent, j'en suis à ma deuxième année aux études universitaires dans ce programme. Ce cours était dans mes options de choix de cours complémentaires à mon cheminement. J'ai décidé de choisir ce cours car ce domaine m'intéresse et il n'a pas été abordé au sein de mon parcours différent de l'informatique traditionnel et du génie logiciel. J'espère en apprendre beaucoup plus sur cette discipline de l'informatique.

- Allo, je m'appelle Mathieu et je suis actuellement en train de faire mon BAC en informatique. Le domaine informatique m'a toujours intéressé donc plusieurs domaines spécialisés tels que l'intelligence artificielle, la cybersécurité et l'infographie pour en nommer quelques-uns. L'infographie m'a attirée en particulier étant donné mon intérêt pour les jeux vidéo, les films d'animation et les effets spéciaux(CGI). J'espère apprendre plus sur comment un ordinateur aujourd'hui est capable de créé, transformé ou simuler différent pour donner l'effet visuel désiré.

- Salut, je m'appelle Vincent, je suis étudiant dans le bac en informatique et je réside à Québec. J'ai beaucoup d'intérêt pour le domaine de la 3D et du jeu vidéo ce qui m'a amené à prendre ce cours dans mon cheminement universitaire. Mon objectif personnel pour ce cours est d'apprendre comment fonctionnent les logiciels utilisés en 3D.

Division des tâches:

Rapport 1:

Fonctionnalités	Membre de l'équipe (principalement responsable)
1.1	Charles Brassard
1.2	Mathieu Chrétien
1.3	Vincent Cousineau
1.4	Anthony Veillet
1.5	Charles Brassard
2.1	Vincent Cousineau
2.2	Anthony Veillet
2.3	Anthony Veillet
2.4	Anthony Veillet
2.5	Charles Brassard
3.1	Charles Brassard
3.2	Vincent Cousineau
3.3	Anthony Veillet
4.1	Mathieu Chrétien
4.2	Mathieu Chrétien
4.3	Mathieu Chrétien
4.4	Charles Brassard
4.5	Mathieu Chrétien
5.1	Vincent Cousineau
5.2	Vincent Cousineau

Rapport 2:

Fonctionnalités	Membre de l'équipe (principalement responsable)
6.1	Mathieu Chrétien
6.2	Mathieu Chrétien
6.3	Mathieu Chrétien
6.5	Mathieu Chrétien
7.1	Anthony Veillet
7.2	Anthony Veillet
7.3	Vincent Cousineau
7.4	Vincent Cousineau
8.1	Charles Brassard
8.2	Charles Brassard
8.3	Charles Brassard
8.4	Charles Brassard
8.5	Charles Brassard
9.1	Vincent Cousineau
9.2	Vincent Cousineau
9.3	Vincent Cousineau
9.4	Mathieu Chrétien
9.5	Anthony Veillet
10.3	Anthony Veillet
10.4	Anthony Veillet